Ing. Max Cerna
Ciencia de Datos
Segundo semestre 2026



Proyecto 2 - Fase 2

¿Quién deja de usar la red?

José Daniel Paz Ortega
carné: 1200022
Jonathan García
carné: 1045222

3 de noviembre de 2026

Introducción

En la Fase 1 cargamos los datos, definimos la etiqueta y probamos la regresión logística. En esta fase probamos ensembles, elegimos el umbral con el escenario de costos, evaluamos una sola vez en el corte B, sacamos los segmentos y dejamos el modelo listo para correr con datos nuevos.

La unidad es la persona y la etiqueta es 1 si no tiene ningún abordaje válido en los 30 días que empiezan en el corte. Comparamos los modelos con la PR-AUC y los mismos 5 folds del corte A, con semilla 42.

Corte	Fecha	Población	Positivos	Cuenta
A	1 de julio	59,491	7,095	7,095 / 59,491 = 0.1193
B	31 de julio	58,113	6,387	6,387 / 58,113 = 0.1099

Los datos son simulados y no corresponden a ningún operador ni usuario real.

2.1 Ensemble y sesgo-varianza

Entrenamos dos ensembles con los mismos folds de la Fase 1. El primero es boosting, un HistGradientBoostingClassifier con 300 árboles de hasta 31 hojas, learning_rate de 0.05, l2 de 1.0 y mínimo 100 personas por hoja. El segundo es bagging, un Random Forest con 300 árboles y mínimo 20 personas por hoja. Los árboles no necesitan variables escaladas, solo el perfil pasa por one-hot dentro del Pipeline.

Modelo	PR-AUC entrenamiento	PR-AUC validación	ROC-AUC validación
Línea base 1: tasa del corte A	0.119	0.119 ± 0.000	0.500
Línea base 2: días desde el último viaje	0.402	0.403 ± 0.014	0.826
Línea base 3: menos viajes en 28 días	0.254	0.254 ± 0.004	0.743
Logística L2, λ = 100	0.416	0.415 ± 0.016	0.831
Logística L1, λ = 10	0.415	0.414 ± 0.016	0.831
Logística Elastic Net, λ = 10	0.415	0.415 ± 0.016	0.831
HistGradientBoosting	0.544	0.426 ± 0.015	0.841
Random Forest	0.533	0.424 ± 0.014	0.841

Sesgo y varianza

Modelo	Diagnóstico	Por qué
Línea base 1	Sesgo alto	Es una constante, no aprende nada.
Línea base 2	Sesgo alto	Una sola variable sin entrenar, 0.402 contra 0.403.
Línea base 3	Sesgo alto	Una sola variable sin entrenar, 0.254 en los dos.
Logística L2	Sesgo alto	0.416 contra 0.415. No se sobreajusta, pero la frontera lineal no saca más.
Logística L1	Sesgo alto	0.415 contra 0.414, y bajar el castigo no ayudó.
Logística Elastic Net	Sesgo alto	0.415 en los dos, igual que las otras logísticas.
HistGradientBoosting	Varianza moderada	Brecha de 0.118, pero con la mejor validación. El mínimo de 100 personas por hoja la mantiene controlada.
Random Forest	Varianza moderada	Brecha de 0.109 y validación apenas abajo del boosting.

Importancia por permutación

La calculamos para el boosting en cada fold de validación, no en entrenamiento, con PR-AUC, y promediamos los 5 folds. El número es cuánto baja la PR-AUC al revolver esa variable.

#	Feature	Baja en PR-AUC	Desviación
1	dias_desde_ultimo_viaje	0.2457	0.0147
2	perfil	0.0152	0.0056
3	tendencia_14d	0.0103	0.0035
4	viajes_14d	0.0090	0.0049
5	viajes_7d	0.0084	0.0066
6	dias_activos_28d	0.0073	0.0051
7	viajes_28d	0.0066	0.0032
8	n_zonas	0.0051	0.0033
9	gasto_28d	0.0045	0.0029
10	prop_hora_pico	0.0033	0.0019

La recencia es la que manda. Sin ella el modelo pierde 0.25 de PR-AUC y las demás aportan poco cada una. Las proporciones por operador y n_operadores no aportaron nada.

Decisión

El boosting va a producción. Le gana a la logística en los 5 folds, por entre 0.007 y 0.016 de PR-AUC, y con el mejor umbral de cada uno deja Q11,480 contra Q11,015 en el corte A. La mejora es pequeña, pero también lo que cuesta: entrena en 2 segundos y se guarda en un

archivo con todo y preprocesamiento. Si la Agencia tuviera que justificar cada contacto con un coeficiente, usaríamos la logística.

2.2 Umbral según costo y test único

Ganancia

Contactar cuesta Q5. Si la persona se iba a ir, la campaña la retiene en el 25% de los casos y una persona retenida vale Q60. Con eso salen los dos casos y la fórmula de ganancia:

```
Contactar a quien se iba:     0.25 × Q60 - Q5 = Q15 - Q5 = +Q10
Contactar a quien no se iba:  -Q5
G = 10 × VP - 5 × FP
```

Para el umbral, a una persona con probabilidad p de irse conviene contactarla si lo que se espera ganar es mayor que el costo:

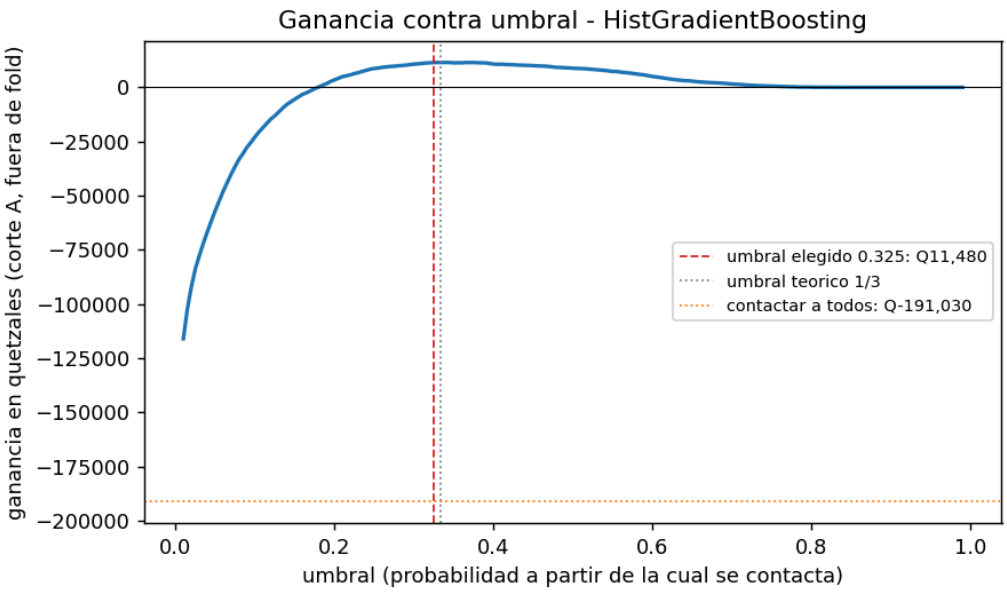
```
p × 15 - 5 > 0
p > 5 / 15 = 1/3
```

Umbral elegido

Barrimos umbrales de 0.01 a 0.99 cada 0.005 con las predicciones de validación cruzada del corte A y elegimos el de mayor ganancia, 0.325. Quedó casi en el 1/3 de la teoría, lo que indica que las probabilidades del boosting están bastante bien calibradas.

Corte A, validación cruzada	Umbral 0.325	Umbral 1/3	Contactar a todos
Contactados	6,320	6,009	59,491
VP	2,872	2,767	7,095
FP	3,448	3,242	52,396
Precisión	0.454	0.460	0.119
Recall	0.405	0.390	1.000
Ganancia	Q11,480	Q11,460	-Q191,030

La cuenta del umbral elegido es 10 × 2,872 - 5 × 3,448 = Q11,480.



Test en el corte B

El 3 de noviembre de 2026 a las 01:44, en el commit 3a6bb7c, entrenamos el boosting con todo el corte A, lo guardamos y lo evaluamos una sola vez en el corte B. Quedó anotado en la bitácora. El script no vuelve a correr si el resultado ya existe o si hay cambios sin commit.

Corte B	Modelo	Línea base de la tasa	Línea base de días sin viajar
PR-AUC	0.386	0.110	0.359

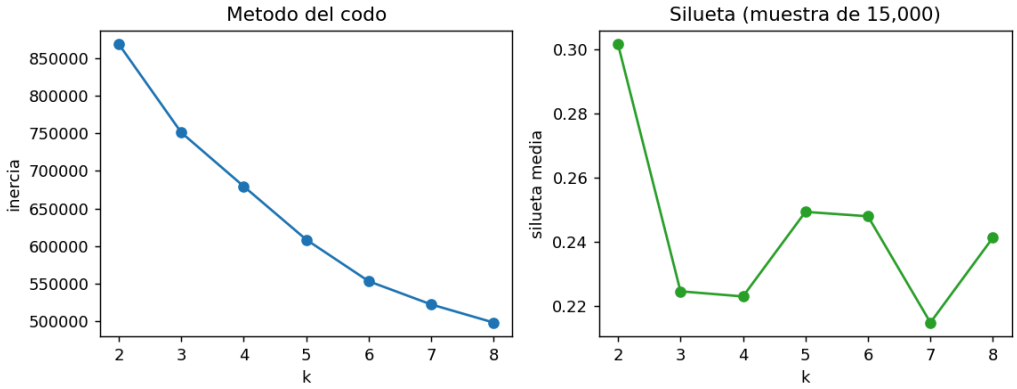
Matriz de confusión, umbral 0.325	Se fue	Siguió viajando
Contactar	VP 2,746	FP 3,982
No contactar	FN 3,641	VN 47,744

La ROC-AUC fue de 0.839, la precisión de 0.408 y el recall de 0.430. La ganancia fue de 10 × 2,746 - 5 × 3,982 = Q7,550.

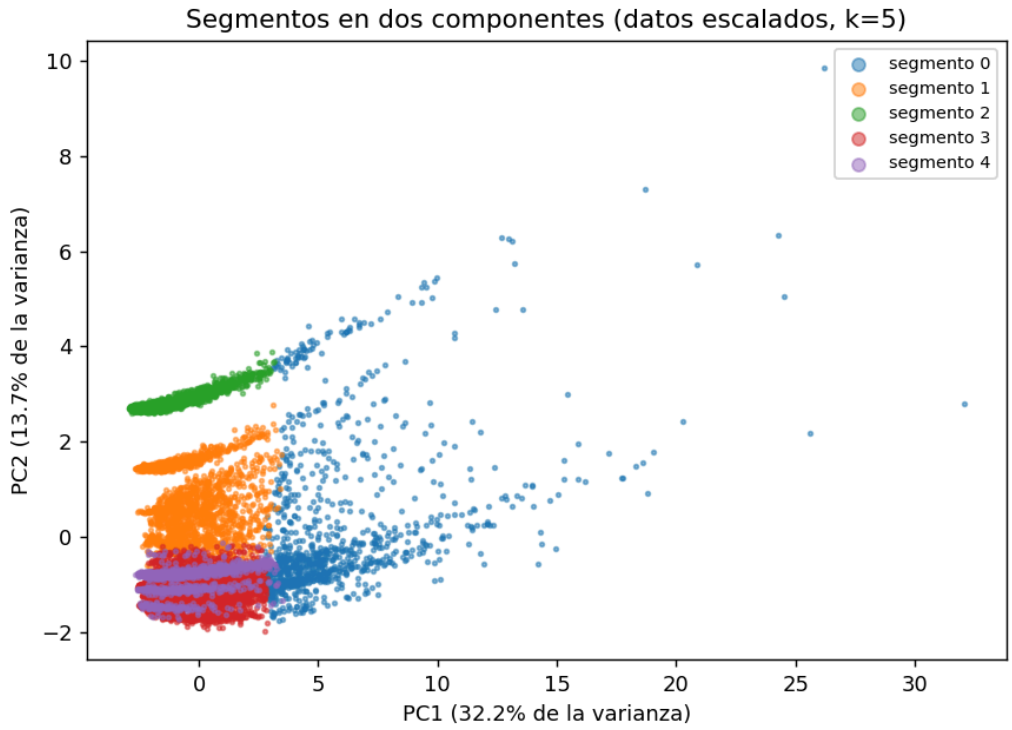
Salió peor que en validación, donde teníamos 0.426 y Q11,480, y no ajustamos nada después de verlo. Creemos que la razón principal es que en B abandona menos gente, 11.0% contra 11.9%. La PR-AUC y la ganancia dependen de cuántos positivos hay, y con menos gente por irse cada contacto acierta menos aunque la lista esté igual de bien ordenada. La ROC-AUC, que no depende de esa tasa, casi no se movió: 0.841 en validación y 0.839 en B. La línea base de días sin viajar también bajó, de 0.403 a 0.359, y contra ella el modelo gana 0.027 en B y 0.023 en validación.

2.3 Segmentación con KMeans y PCA

Corrimos KMeans con las 18 features numéricas del corte A ya escaladas. El perfil no entra porque es categórico. Probamos k de 2 a 8. La silueta más alta fue k = 2, con 0.302, pero solo separa a los que viajan muchísimo del resto y con eso no se puede pensar una campaña. El siguiente pico fue k = 5, con 0.249, justo donde el codo se empieza a aplanar, y por eso nos quedamos con 5.



Con los datos escalados proyectamos los segmentos en dos componentes principales, que explican el 45.9% de la varianza: 32.2% la primera y 13.7% la segunda.



#	Nombre	Tamaño	Viajes 28d	Días activos	Días desde último viaje	Operador principal	Abandono
0	Viajeros intensivos	5,461	53	21	0.3	Transurbano 70% y otro	0.019
1	Usuarios de MetroRiel	10,082	9	7	2.6	MetroRiel 80%	0.125
2	Usuarios de Aerómetro	10,028	6	5	3.7	Aerómetro 100%	0.145
3	Usuarios de Transurbano	25,482	9	7	2.6	Transurbano 100%	0.118
4	Usuarios de Transmetro	8,438	9	7	2.6	Transmetro 100%	0.149

Los valores son medianas. La tasa de abandono solo describe dónde se concentra, no prueba que el segmento lo cause. El segmento no se usa como feature del modelo.

Acciones de retención

A los usuarios de Transmetro, que tienen la tasa más alta, les daríamos una recarga de cortesía que se active solo si vuelven a viajar la semana siguiente, con un aviso de las frecuencias de sus líneas.

A los de Aerómetro, un transbordo sin costo hacia Transmetro o Transurbano, para que el teleférico no sea un viaje aislado. En este segmento no podemos saber si la persona sigue viajando con otra tarjeta.

A los de MetroRiel, un pase semanal con descuento, porque viajan casi siempre en hora pico.

En los viajeros intensivos no gastaríamos presupuesto, porque casi no abandonan.

2.4 MLOps: predecir, guardar y documentar

El script recibe la carpeta de datos y la fecha de corte:

```
python predecir.py --datos datos_red --fecha-corte 2026-08-30
```

Carga los datos con el pipeline desde cero, calcula las features del corte, carga el modelo y escribe predicciones/predicciones_2026-08-30.csv con unidad_id, tarjetas separadas por punto y coma, puntaje y contactar. No vuelve a entrenar.

El modelo está en modelos/modelo_abandono_v1.joblib. El archivo guarda el Pipeline completo, el umbral, las columnas, el commit y la versión de scikit-learn. Las versiones de las librerías están fijas en requirements.txt, la semilla es 42 y los pasos para correrlo están en el README. La ficha del modelo va en un documento aparte.

En el corte C evaluamos a 57,701 personas y el modelo recomienda contactar a 7,872.

Ensayo de la prueba de invariancia

No tenemos septiembre, pero con el corte B se puede hacer la misma prueba. Predijimos B con los archivos recortados al 31 de julio y con los datos completos, que traen 30 días más. Los dos CSV salieron idénticos, con 58,113 filas y ninguna diferencia. Se repite con python -m ml.prueba_invariancia.

En ese ensayo nos salió un problema. Dos corridas de predecir.py con los mismos datos daban 2,759 puntajes distintos. DuckDB suma en paralelo y las tarifas con decimales, como Q1.30, cambiaban en el último decimal según el orden de los hilos. El boosting parte las variables en intervalos y esa diferencia tan pequeña movía a algunas personas de un intervalo a otro. Lo arreglamos sumando el gasto en centavos enteros. No reentrenamos ni repetimos el test en B, porque el dato es el mismo y solo cambió cómo se suma. Después de eso, tres corridas seguidas dieron el mismo archivo.

Métricas del proyecto

Categoría	Resultado
Problema	A: 7,095 / 59,491 = 0.1193. B: 6,387 / 58,113 = 0.1099. C: 57,701 personas, etiqueta oculta.
Validación	PR-AUC de 0.426 ± 0.015 del boosting, contra 0.403 ± 0.014 de la línea base de días sin viajar y 0.119 ± 0.000 de la tasa.
Decisión	Umbral 0.325, precisión 0.454, recall 0.405 y Q11,480 en A. Contactar a todos daría -Q191,030.
Test	Corte B, 3 de noviembre de 2026, commit 3a6bb7c. PR-AUC de 0.386, contra 0.359 de días sin viajar y 0.110 de la tasa. Ganancia de Q7,550.
Fuga	Prueba de recorte con 0 diferencias en Silver y en crudo. Invariancia ensayada con B, archivos idénticos. La prueba con 30 días más se hace en la defensa.
Rendimiento	Carga desde cero entre 25 y 67 segundos. Features de los 3 cortes entre 4 y 10 segundos. Validación cruzada del boosting 6 segundos. Entrenamiento final 2 segundos. predecir.py 38 segundos con el pipeline incluido y 4 sin él.